

Curiosity Driven Knowledge Retrieval for Mobile Agents

Sijia Li*
sijialiyabing@gmail.com
Shanghai University of Engineering
Science
Shanghai, China

Xiaoyu Tan*
txywilliam1993@outlook.com
National University of Singapore
Singapore

Shahir Ali
shahir@droidrun.ai
Droidrun
Osnabrück, Germany

Niels Schmidt
niels@droidrun.ai
Droidrun
Osnabrück, Germany

Gengchen Ma
m320123320@sues.edu.cn
Shanghai University of Engineering
Science
Shanghai, China

Xihe Qiu†
qiuxihe1993@gmail.com
Shanghai University of Engineering
Science
Shanghai, China

Abstract

Mobile agents have made progress toward reliable smartphone automation, yet performance in complex applications remains limited by incomplete knowledge and weak generalization to unseen environments. We introduce a curiosity driven knowledge retrieval framework that formalizes uncertainty during execution as a curiosity score. When this score exceeds a threshold, the system retrieves external information from documentation, code repositories, and historical trajectories. Retrieved content is organized into structured *AppCards*, which encode functional semantics, parameter conventions, interface mappings, and interaction patterns. During execution, an enhanced agent selectively integrates relevant *AppCards* into its reasoning process, thereby compensating for knowledge blind spots and improving planning reliability. Evaluation on the AndroidWorld benchmark shows consistent improvements across backbones, with an average gain of six percentage points and a new state of the art success rate of 88.8% when combined with GPT-5. Analysis indicates that AppCards are particularly effective for multi step and cross application tasks, while improvements depend on the backbone model. Case studies further confirm that AppCards reduce ambiguity, shorten exploration, and support stable execution trajectories. Task trajectories are publicly available at <https://lisalsj.github.io/Droidrun-appcard/>.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence; Planning and scheduling; Knowledge representation and reasoning.**

Keywords

Mobile Agents, Multimodal Large Language Models, Knowledge Retrieval, Task Automation

*Both authors contributed equally to this research.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792257>

ACM Reference Format:

Sijia Li, Xiaoyu Tan, Shahir Ali, Niels Schmidt, Gengchen Ma, and Xihe Qiu. 2026. Curiosity Driven Knowledge Retrieval for Mobile Agents. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792257>

1 Introduction

Mobile agents aim to deliver convenient and intelligent automation on smartphones. Given a natural language instruction, an agent infers user intent, constructs an execution plan, and orchestrates actions across installed applications to accomplish the specified goal. Advances in multimodal large language models [37] have markedly improved interaction competence, task understanding, and coverage of application ecosystems, indicating strong potential for intelligent interaction and everyday automation [22]. Despite this progress, practical deployments reveal important limitations. When interacting with complex applications or with environments that have not been encountered before, agents often lack application specific functional knowledge. This knowledge incompleteness produces planning errors, incorrect invocation of applications or tools, and reduced task success, and it limits generalization to new tasks and applications.

Existing approaches to strengthening task execution in mobile agents fall into two broad lines [29]. The first line emphasizes application oriented end to end training in which large scale interaction data from one or a small number of target applications is used for supervised fine tuning SFT [2] or reinforcement fine tuning RFT [36]. These methods yield substantial gains within the environments used for training yet they transfer poorly to previously unseen applications. Typical failure modes include overfitting to specific interface layouts and action vocabularies, mismatch between learned policies and new widget taxonomies, and brittle behavior under distribution shift. A complementary line focuses on framework level improvements that aim to build stable and general agent architectures for planning and reasoning [10, 11]. Although these designs improve the handling of complex interactive tasks and support longer execution horizons, they usually encode only shallow or implicit knowledge of application functionality. As a result the agent often issues erroneous API or function calls, including incorrect entry points and misparameterized arguments,

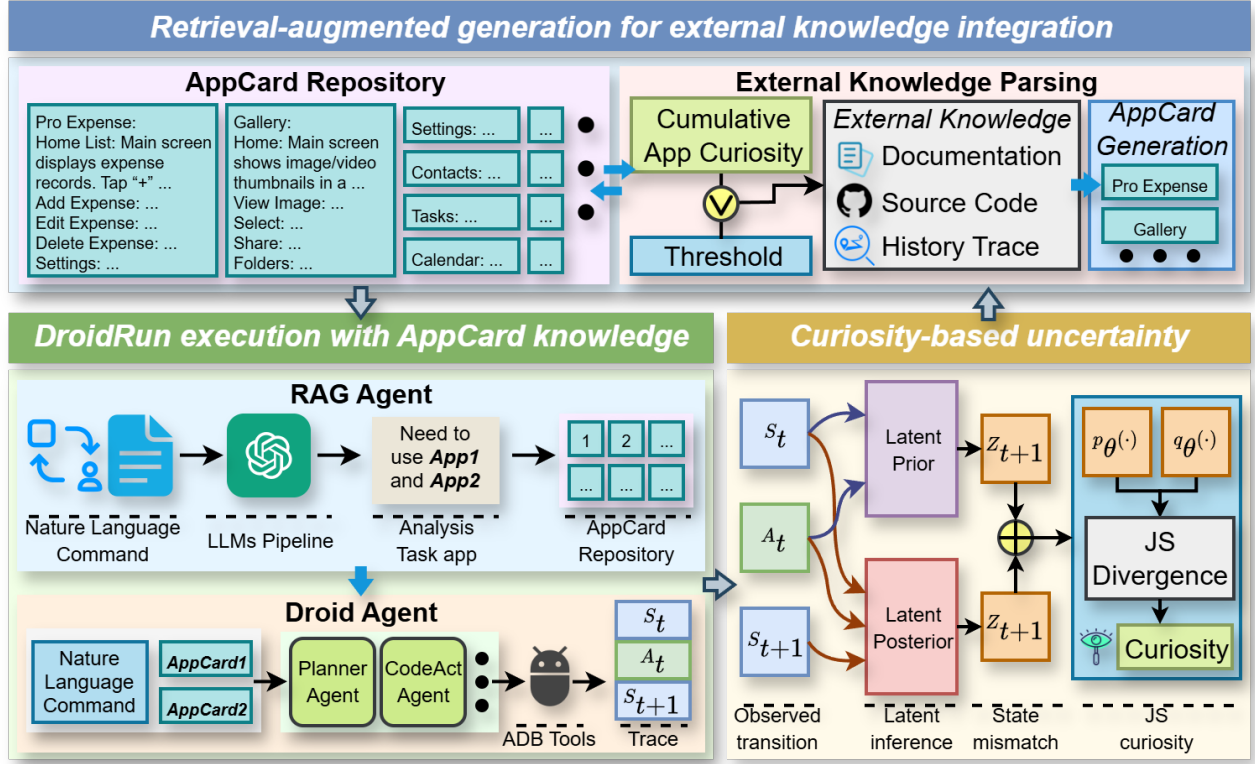


Figure 1: Overall framework of the curiosity driven knowledge retrieval system for mobile agents. Task execution is guided by AppCards. Uncertainty estimation produces curiosity signals that activate external retrieval. Retrieved knowledge is consolidated to update AppCards and the updated AppCards are reintegrated into the execution pipeline.

which propagate through the interaction sequence and ultimately limit overall task completion.

In this work a curiosity driven knowledge retrieval framework is introduced to strengthen task execution of mobile agents in complex application scenarios. A curiosity score is formalized to quantify the agent’s insufficient understanding during application interactions as uncertainty over functional knowledge. When this score for a given application exceeds a predefined threshold [12], the system initiates external knowledge retrieval [13]. The curiosity gate serves as a resource allocation mechanism that bounds retrieval frequency, preserves context window budget, and concentrates attention on states with the largest epistemic deficit. The retrieval process extracts salient information from application documentation and source repositories and organizes the material into structured *AppCards*. During execution the agent analyzes the task, identifies the relevant applications, and invokes the corresponding *AppCards* so that application knowledge is incorporated into reasoning and decision making. Because *AppCards* are modular and version aware, only the fragments required by the current step are injected, which reduces spurious associations and improves stability under user interface changes. This pipeline yields a finer grained

representation of application functionality at the framework level and improves planning reliability, automation efficiency, and task success in challenging scenarios.

Specifically, our contributions can be summarized as follows:

- (1) A curiosity driven knowledge augmentation framework is presented, which quantifies insufficient understanding as a curiosity score and uses this signal as a principled trigger for external knowledge retrieval.
- (2) A structured representation termed *AppCards* is developed to consolidate knowledge from documentation and repositories, enabling efficient integration and enhancing fine grained understanding of application functionalities.
- (3) Comprehensive evaluation on the AndroidWorld benchmark [27] with the DroidRun agent demonstrates consistent gains in task success and robustness, with further validation across unseen applications and multiple backbone models.

2 Related Work

2.1 Mobile Agents and GUI Automation

In the study of GUI automation and mobile agents, early approaches primarily relied on scripting and demonstration-driven strategies.

RecurBot [14] enabled automatic completion of repetitive operations by identifying loop patterns within demonstrations, while testing frameworks such as Appium and SilkTest [3, 24] provided script-based cross-platform interaction capabilities. With the advancement of multimodal models, research has increasingly shifted toward unified frameworks that integrate visual and linguistic information. Aria-UI [33] achieved vision-only grounding through large-scale synthetic data and context modeling, and SeeClick [5] introduced GUI pre-training together with the ScreenSpot benchmark. UI-TARS [26] and AndroidGen [16] explored instruction-to-element mapping in diverse application scenarios using multimodal models, whereas GUI-Explorer [31] improved cross-task generalization by combining screenshots with accessibility trees. Building on these efforts, more recent studies emphasize the design of agent frameworks; for example, MobileUse [18] employs a hierarchical reflection mechanism to enhance stability and adaptability in task execution. Collectively, these works have advanced mobile agents toward more reliable understanding of natural language instructions and robust interaction with user interfaces across applications and platforms.

2.2 Knowledge Augmentation and Reasoning Frameworks

In the context of knowledge augmentation for agents, LLM-Explorer introduces a knowledge maintenance-centric framework that leverages large language models to construct and update abstract interaction graphs, thereby achieving high interface coverage while substantially reducing inference calls and associated costs [38]. However, its effectiveness remains limited in scenarios requiring cross-application transitions and state sharing. Recent work has emphasized large-scale interaction data pretraining to enhance the generality and transferability of agents. AutoGLM develops a deployable foundation agent system across representative GUI domains such as browsers and mobile applications, integrating perception, planning, and execution modules to strengthen cross-application capabilities [20], while GUI-Owl-32B/7B serves as a foundation model that demonstrates robust end-to-end performance and portability across both desktop and mobile environments [35]. Moreover, although SymAgent is not designed specifically for mobile agents, its neural symbolic self-learning framework treats knowledge graphs as dynamic reasoning environments. By combining a planner and an executor architecture with external retrieval augmentation, it improves complex task decomposition and execution stability [19]. This approach provides useful insights into knowledge augmentation and reasoning for mobile agents in scenarios that involve complex reasoning and long-horizon tasks.

2.3 Curiosity Driven Knowledge Exploration

Curiosity in reinforcement learning is commonly viewed as an intrinsic mechanism that drives agents to explore uncertain states [28]. A representative approach is the predictive model of Pathak et al., where large prediction errors indicate poorly captured dynamics and thus yield intrinsic rewards that encourage exploration [25]. Recent work extends this idea to large language models, using RL-style formulations to acquire missing knowledge during reasoning [34]. For example, WorldLLM combines Bayesian inference with

curiosity-driven exploration to iteratively generate and validate interpretable world hypotheses [17], while CDE defines curiosity rewards from perplexity-based actor signals and uncertainty estimated from multi-critic variance to improve RLVR training [7]. More broadly, uncertainty-guided mechanisms have been used to steer selective decision-making under distribution shift [9]. Inspired by this view, we use curiosity as a lightweight trigger for targeted knowledge acquisition: when the agent is uncertain in a previously unseen application, it initiates external retrieval and constructs an AppCard to reduce future uncertainty with minimal manual effort.

3 Methods

3.1 Problem Formulation

We formulate mobile-agent task execution as a variant of the Markov Decision Process (MDP) [4], represented by the five-tuple

$$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, U, \tau, \gamma\}. \quad (1)$$

Here, $\mathcal{S}$ denotes the state space, corresponding to the mobile interface observed by the agent. $\mathcal{A}$ represents the action space, including operations such as *tap_by_index*, *tap_by_text*, *input_text*, and *swipe*. Different from the classical MDP, we introduce U as a quantitative measure of the agent’s epistemic uncertainty about application functionalities, serving as the intrinsic signal for curiosity-driven exploration. τ denotes the exploration threshold: once accumulated uncertainty exceeds this value, the agent triggers external knowledge acquisition to refine its comprehension. Finally, $\gamma \in (0, 1]$ is the discount factor that balances immediate and long-term rewards.

This formulation highlights the dual role of uncertainty. It reflects the agent’s cognitive limitations and governs the initiation of external knowledge retrieval, thereby enhancing robustness and generalization (Figure 1).

3.2 Curiosity-Driven Information Gain Estimation

When vision-language models act as agents to accomplish tasks, it is crucial to quantify their insufficiency in understanding the application environment in order to trigger external knowledge retrieval. Inspired by the concept of **Latent Bayesian Surprise (LBS)** [23], we treat the divergence between the prior prediction and the posterior observation as the principal signal for measuring uncertainty. This divergence reflects the information gain acquired during interactions and can be interpreted as a curiosity signal that prompts the agent to proactively initiate external knowledge retrieval when facing knowledge gaps. This formulation is also aligned with recent advances CDE [7], which demonstrate that explicitly modeling uncertainty as an intrinsic signal can effectively guide exploration and improve reasoning performance.

3.2.1 Latent State Distribution Modeling. Given the current interface state s_t and the executed action a_t , the model generates a prior predictive distribution P for the description of the next interface. After observing the actual subsequent state s_{t+1} , the model produces a posterior distribution Q for the corresponding description. At each decoding step t , only the Top K candidate tokens are retained, and

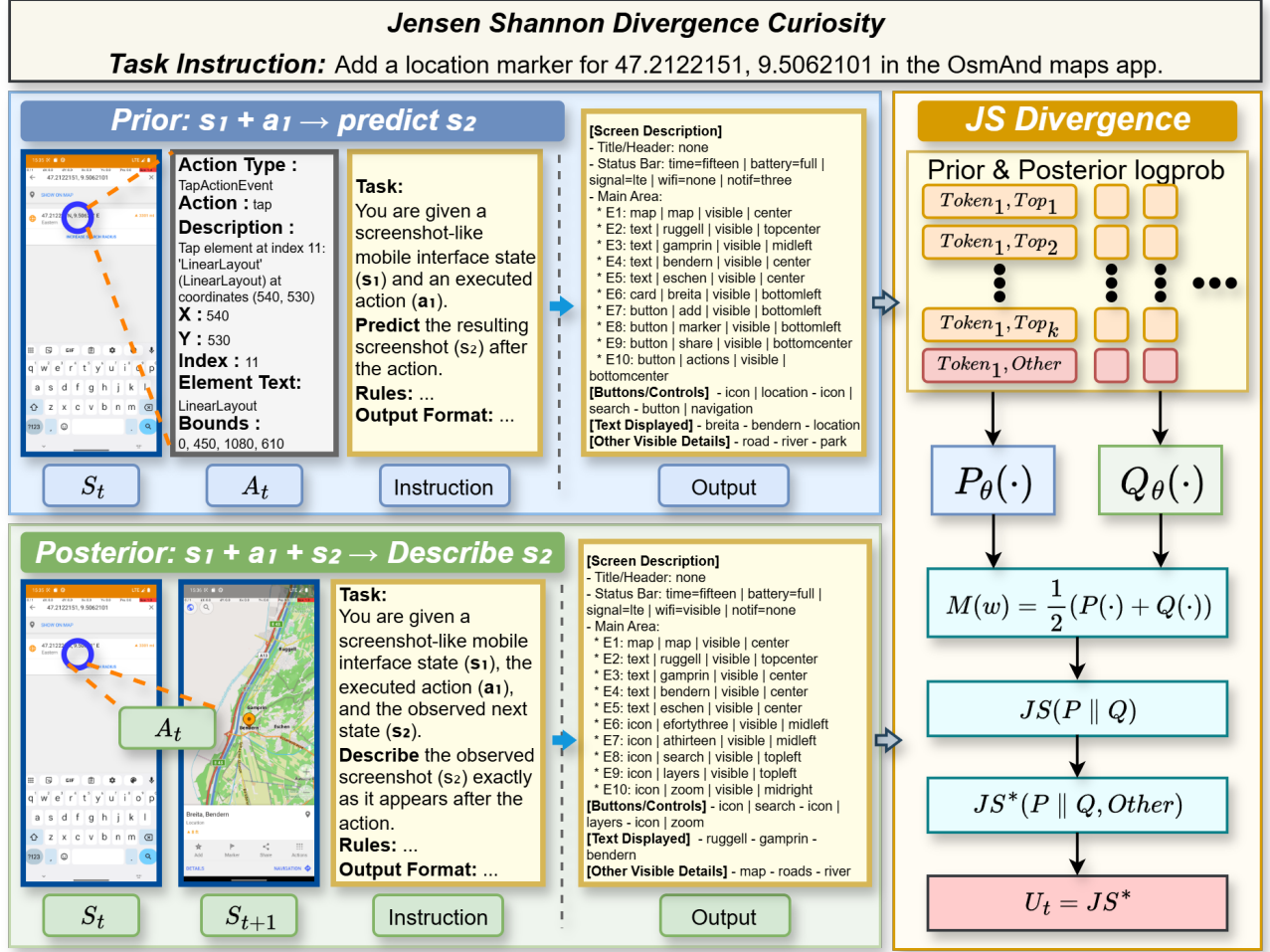


Figure 2: JS-divergence based curiosity estimation. The agent predicts the next interface state from the current state and action as a prior distribution, then observes the next state as a posterior distribution. The divergence between these distributions is measured with a tail adjusted Jensen Shannon divergence, yielding an information gain signal quantifying curiosity.

the remaining probability mass is aggregated into a residual bucket denoted as OTHER. Formally,

$$p_{t,i} = \exp(\ell_{t,i}), \quad q_{t,i} = \exp(\hat{\ell}_{t,i}), \quad i = 1, \dots, K \quad (2)$$

where $\ell_{t,i}$ denotes the prior log probability of the i th token at decoding step t , and $\hat{\ell}_{t,i}$ denotes the posterior log probability of the i th token at the same step after incorporating s_{t+1} . The residual probabilities at each decoding step are defined as

$$p_{t,other} = 1 - \sum_{i=1}^K p_{t,i}, \quad q_{t,other} = 1 - \sum_{i=1}^K q_{t,i} \quad (3)$$

Each output token distribution is then normalized so that the $K + 1$ dimensional probability vector consisting of the Top K tokens and the residual probability sums to one. By aggregating across all decoding positions, we define a union set of length T . For any token

w , the probabilities are accumulated and normalized to obtain the final prior distribution and posterior distribution as

$$P(w) = \frac{\sum_{t=1}^T p_t(w)}{T}, \quad Q(w) = \frac{\sum_{t=1}^T q_t(w)}{T} \quad (4)$$

This procedure yields global prior and posterior distributions that capture token-level semantics across decoding steps. Figure 3 provides an illustrative example of such aggregated distributions, offering an intuitive view of the differences between P and Q .

3.2.2 Information Gain Estimation. Within a Bayesian perspective, the information acquired by the agent during the interaction $s_t, a_t \rightarrow s_{t+1}$ is measured by the divergence between the prior and the posterior distributions

$$I(z_{t+1}; s_{t+1} | s_t, a_t) \approx D_{KL}(P \parallel Q) \quad (5)$$

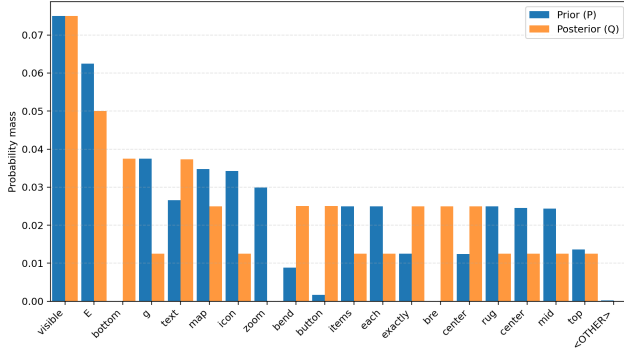


Figure 3: An example of aggregated global semantic distributions with Top-19 tokens and the residual OTHER category

Here z_{t+1} denotes a latent representation at time $t+1$ learned by the model, which encodes the semantic content of the next interface state. A larger Kullback Leibler (KL) divergence indicates a substantial mismatch between prediction and observation, hence the interaction provides new information and reveals a gap in the model’s internal knowledge.

To obtain a symmetric and bounded measure, we adopt the base two Jensen Shannon (JS) divergence

$$JS(P \parallel Q) = \frac{1}{2} \left[\sum_w P(w) \log_2 \frac{P(w)}{M(w)} + \sum_w Q(w) \log_2 \frac{Q(w)}{M(w)} \right] \quad (6)$$

where

$$M(w) = \frac{1}{2} (P(w) + Q(w)) \quad (7)$$

To better reflect the uncertainty carried by the probability mass outside the Top K candidates, we introduce a tail adjustment based on the residual bucket OTHER

$$JS^*(P \parallel Q) = JS(P \parallel Q) + \lambda \cdot \frac{1}{2} (P(\langle OTHER \rangle) + Q(\langle OTHER \rangle)) \quad (8)$$

with a tunable coefficient $\lambda > 0$. The adjusted divergence accounts for the principal token distributional shift and explicitly incorporates the uncertainty from the long tail.

We finally use the adjusted divergence as the single step information gain

$$I(z_{t+1}; s_{t+1} \mid s_t, a_t) \approx JS^*(P \parallel Q) \quad (9)$$

A larger value implies a greater deficiency in the model’s understanding of the application and thus a stronger curiosity for exploration. The computation pipeline is illustrated in Figure 2, which depicts the transformation from prior and posterior probabilities to the JS divergence and its tail adjusted form. In addition, Figure 4 presents an example of token-wise contributions to the divergence, highlighting which semantic elements dominate the difference between P and Q .

3.2.3 Cumulative Uncertainty. To reduce the randomness of single step information gain estimates, we introduce a discounted accumulation at the application level

$$U(\text{app}) = \sum_{t \in T_{\text{app}}} \gamma_t \cdot JS^*(P_t \parallel Q_t) \quad (10)$$

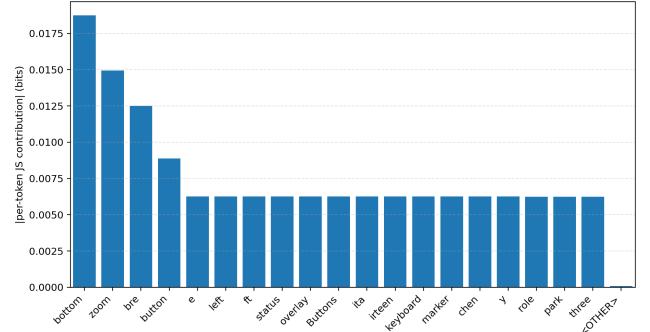


Figure 4: An example of token-level contributions to the adjusted JS divergence, showing the top 19 tokens and the residual OTHER category.

where γ_t is a weight reflecting both task difficulty and temporal decay. Once the cumulative uncertainty $U(\text{app})$ exceeds a predefined threshold τ , the system proactively initiates external knowledge retrieval. Relevant information is gathered from external resources such as documentation, source code repositories, and historical trajectories, and is injected in a structured form to compensate for the model’s knowledge blind spots.

3.3 Retrieving and Embedding External Knowledge

When the cumulative uncertainty $U(\text{app})$ exceeds the predefined threshold τ , the agent proactively triggers external knowledge retrieval in order to compensate for its insufficient understanding of application functionalities. Unlike approaches that rely solely on internal parametric knowledge, this mechanism explicitly incorporates external resources into the reasoning loop, thereby providing additional structured support in high uncertainty scenarios.

We consider three heterogeneous sources of knowledge. The first is online documentation D_{web} , which includes developer manuals, API documentation, and user guides. The second is source code and version history D_{git} , which covers function definitions, dependency structures, and development logs. The third is historical interaction trajectories D_{traj} , which record execution paths and experiential traces. Once retrieval is triggered, the system uses the uncertainty signal $U(\text{app})$ together with the threshold τ to form targeted queries, and consolidates candidate knowledge units from these multimodal sources

$$\text{Card}(\text{App}) = \text{Retrieve}(D_{\text{web}} \cup D_{\text{git}} \cup D_{\text{traj}}, U(\text{app}), \tau) \quad (11)$$

To efficiently integrate the retrieved knowledge, we employ a structured representation referred to as AppCards. Each AppCard is constructed for a single application with the objective of compressing unstructured information into modular knowledge units that are convenient for reasoning. A typical AppCard consists of four components. The first component is functional semantics, which describes the purpose and applicable scope of API calls. The second is input and output constraints, which specify parameter types, return values, and boundary conditions. The third is the mapping between interface elements and functionalities, which aligns user interface

Table 1: Comparison on *AndroidWorld*. SR denotes task success rate (%).

Method	Model	Screen Representation	SR
Aria-UI [33]	GPT-4o + Aria-UI	Screenshot	44.8
UI-TARS [26]	UI-TARS	Screenshot	46.6
AndroidGen [16]	GPT-4o	A11y tree	46.8
GUI-Explorer [31]	GPT-4o	Screenshot + A11y tree	47.4
Agent S2 [1]	Agent S2	Screenshot	54.3
V-Droid [6]	V-Droid (Llama-8B)	A11y tree	59.5
JT-GUIAgent-V1 [15]	JT-GUIAgent-V1	Screenshot	60.0
JT-GUIAgent-V2 [30]	JT-GUIAgent-V2	Screenshot	67.2
Mobile-Agent-v3 [35]	GUI-Owl-32B	Screenshot	73.3
MobileUse-v2 [18]	Hammer-UI-32B	Screenshot + A11y tree	75.0
LX-GUIAgent [21]	LX-GUIAgent	Screenshot + A11y tree	79.3
AutoGLM-Mobile [32]	AutoGLM-Mobile	Screenshot + A11y tree	80.2
mobile-use [18]	Llama 4-scout, Gemini 2.5 pro, GPT-5 nano	Screenshot + A11y tree	84.5
DroidRun(v0.3.3) [8]	Gemini-2.5-Pro	Screenshot + A11y tree	63.0
DroidRun(v0.3.9)	GPT-5, Gemini 2.5 Pro	Screenshot + A11y tree	78.4
DroidRun(v0.3.3)+AppCards (ours)	Gemini-2.5-Pro	Screenshot + A11y tree	69.0
DroidRun(v0.3.9)+AppCards (ours)	Grok-4-Fast	Screenshot + A11y tree	60.3
DroidRun(v0.3.9)+AppCards (ours)	Gemini-2.5-Pro	Screenshot + A11y tree	71.6
DroidRun(v0.3.9)+AppCards (ours)	GPT-5	Screenshot + A11y tree	88.8

Results are collected up to October 1, 2025, based on the *AndroidWorld Leaderboard*, where our proposed method holds the **top position**

components such as button texts with their triggered functions. The fourth is a set of common interaction patterns and exception handling strategies. Through a unified template design, AppCards achieve multi-source coverage while ensuring interpretability and reusability of the knowledge representation.

During task execution, the agent first identifies which applications need to be invoked, then retrieves the corresponding AppCards and injects them into the system prompt. In this manner, uncertainty is operationalized as a curiosity signal, with information gain measured through the divergence between prior and posterior distributions. Once cumulative uncertainty exceeds the threshold, the system activates external retrieval and incorporates the structured AppCard representations into the reasoning process, thereby mitigating knowledge deficiencies and enhancing the agent’s generalization capability.

4 Experimental

4.1 Experimental Settings

Benchmark. Experiments are conducted on the *AndroidWorld* benchmark, which includes 116 interactive tasks across about twenty widely used Android applications. Tasks are instantiated from parameterized templates to evaluate both proficiency and generalization. The benchmark spans scheduling, configuration, note taking, finance, messaging, media, and web navigation. Tasks are grouped into three difficulty levels: 61 easy tasks with short single-app interactions, 36 medium tasks involving multi-step creation or editing,

and 19 hard tasks requiring extended reasoning or cross-app coordination. We report task success rate, defined as completing the goal within a fixed interaction budget. Unless otherwise specified, agents observe both screenshots and accessibility trees at each step.

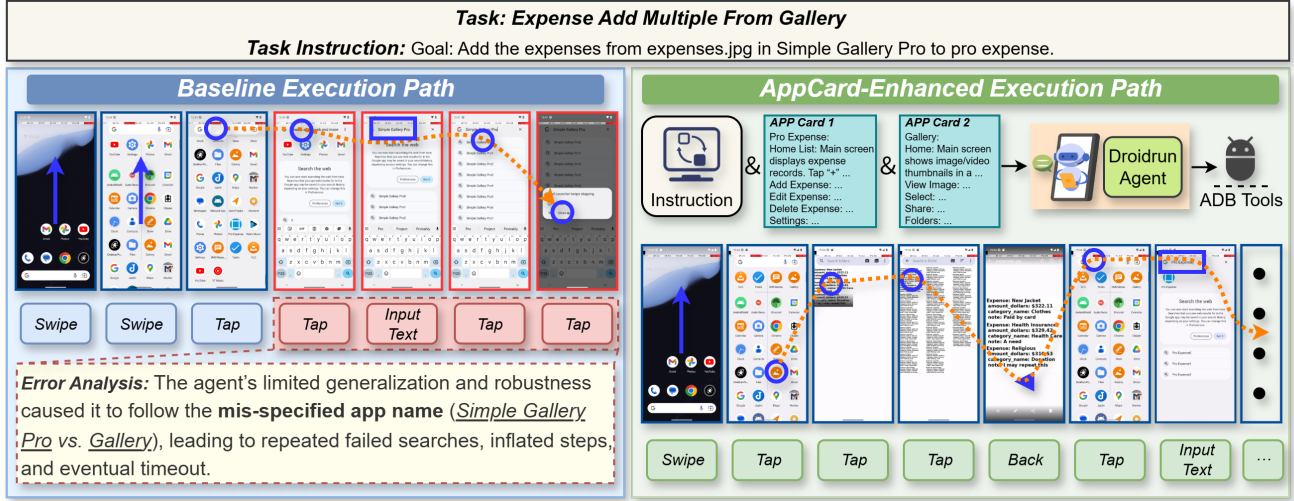
Implementation details. All evaluations use the open-source *DroidRun* framework for stable execution on *AndroidWorld*. We consider two releases (v0.3.3 and v0.3.9) to assess robustness to infrastructure changes. Unless otherwise specified, Gemini 2.5 Pro is the default backbone (temperature 0.3); we also report results with GPT 5 and Grok 4 fast under v0.3.9. Our system augments DroidRun with a curiosity-driven knowledge retrieval module: a curiosity score estimates uncertainty about functional knowledge and triggers external retrieval when it exceeds a fixed threshold. Retrieved information from application documentation and source repositories is consolidated into structured *AppCards* and selectively incorporated into reasoning and action planning. All methods use the same step budget, observation modality, and environment configuration for fair comparison.

4.2 Results

Table 1 presents the overall comparison on the *AndroidWorld* benchmark. Compared with existing state-of-the-art methods, DroidRun with AppCards using GPT 5 in version v0.3.9 achieves a success rate of **88.8%**, surpassing the previously reported **84.5%** of mobile use and establishing a new publicly available best result. In terms of relative improvement, GPT 5 increases from **84.5%** to **88.8%**, corresponding to an absolute gain of **4.3** percentage points and a

Table 2: Performance comparison of DroidRun and DroidRun+AppCards across difficulty levels. The numbers indicate successes out of total tasks, with success rates and relative improvements.

Model	Method	Easy (61)		Medium (36)		Hard (19)	
		Succ.	Rate	Succ.	Rate	Succ.	Rate
Gemini-2.5-Pro	DroidRun(v0.3.3)	48	78.7%	20	55.6%	5	26.3%
Gemini-2.5-Pro	DroidRun(v0.3.3)+AppCards	51	83.6% ($\uparrow 4.9\%$)	22	61.1% ($\uparrow 5.5\%$)	7	36.8% ($\uparrow 10.5\%$)
Grok-4-fast	DroidRun(v0.3.9)	43	70.5%	18	50.0%	10	52.6%
Grok-4-fast	DroidRun(v0.3.9)+AppCards	44	72.1% ($\uparrow 1.6\%$)	20	55.6% ($\uparrow 5.6\%$)	6	31.6% ($\downarrow 21.0\%$)
Gemini-2.5-Pro	DroidRun(v0.3.9)	48	78.7%	19	52.8%	9	47.4%
Gemini-2.5-Pro	DroidRun(v0.3.9)+AppCards	51	83.6% ($\uparrow 4.9\%$)	22	61.1% ($\uparrow 8.3\%$)	10	52.6% ($\uparrow 5.2\%$)
GPT-5	DroidRun(v0.3.9)	55	90.2%	32	88.9%	11	57.9%
GPT-5	DroidRun(v0.3.9)+AppCards	56	91.8% ($\uparrow 1.6\%$)	32	88.9% ($\uparrow 0.0\%$)	15	78.9% ($\uparrow 21\%$)

**Figure 5: Case study of the task Expense Add Multiple From Gallery. The baseline path on the left fails due to application name ambiguity, while the AppCard enhanced path on the right leverages structured knowledge to enable stable and successful task execution.**

relative increase of approximately 5.1%. For Gemini 2.5 Pro, the performance in version v0.3.3 improves from 63.0% to 69.0%, and in version v0.3.9 improves from 65.5% to 71.6%. Both versions yield gains of around 6 percentage points, with relative improvements in the range of 9 to 10%. In contrast, Grok 4 fast decreases from 61.2% to 60.3% in version v0.3.9, showing no benefit from AppCards.

Taken together, these results indicate that the effectiveness of AppCards varies across models and versions. When the backbone model has strong reasoning and knowledge integration capabilities, AppCards consistently translate into performance improvements, with GPT 5 achieving breakthrough results. For weaker or stylistically mismatched models, however, AppCards may not be fully utilized and can even introduce negative effects. This suggests that AppCards not only contribute to performance enhancement but also expose critical interactions between external knowledge and model characteristics, offering insights for designing more robust knowledge injection mechanisms.

4.3 Ablation Study

Table 2 reports detailed performance across different task difficulty levels. In version v0.3.3, Gemini 2.5 Pro combined with AppCards improves from 78.7% to 83.6% on easy tasks, from 55.6% to 61.1% on medium tasks, and from 26.3% to 36.8% on hard tasks. Although improvements appear across all levels, the most substantial gain is observed in hard tasks, with an increase of 10.5 percentage points, highlighting the value of AppCards in guiding models through complex task structures. In version v0.3.9, GPT 5 exhibits even more pronounced improvements. Easy tasks remain nearly unchanged, moving from 90.2% to 91.8%. Medium tasks remain stable at 88.9%. Hard tasks, however, increase dramatically from 57.9% to 78.9%, yielding a 21.0 percentage point improvement. This underscores the critical role of AppCards in bridging knowledge gaps under complex conditions. Gemini 2.5 Pro in the same version shows consistent improvements of 4.9, 8.3, and 5.2 percentage points across easy,

medium, and hard tasks, respectively, reflecting balanced benefits. By contrast, Grok 4 fast shows modest gains on easy and medium tasks but experiences a sharp decline on hard tasks, dropping from 52.6% to 31.6%, a decrease of 21.0 percentage points, indicating that its knowledge integration ability is insufficient to leverage AppCards in demanding scenarios.

Overall, these stratified experiments demonstrate that AppCards provide the greatest benefit in complex tasks, substantially enhancing long-horizon reasoning and cross-application operations. At the same time, the experiments reveal clear divergences among models. Strong backbones are able to reliably absorb external knowledge and achieve significant gains, whereas weaker models may encounter instability. This contrast shows that AppCards not only improve average performance but also provide concrete evidence for understanding the mechanisms of interaction between external knowledge and model capabilities.

5 Case Study

Task and Challenge: The examined task requires the agent to transfer the file *expenses.jpg* from *Simple Gallery Pro* into *Pro Expense*. This operation involves coordinated interactions across multiple applications and therefore places high demands on the model’s ability to disambiguate entities, identify functional entry points, and reason over inter-application paths. Baseline experiments reveal that in the absence of external knowledge the model is unable to complete the task reliably. The agent frequently confuses the system gallery with the intended application, repeatedly searches within the app drawer, and generates redundant or erroneous actions. These behaviors accumulate until the interaction budget is exhausted, highlighting fundamental deficiencies in semantic grounding and strategic planning across heterogeneous applications.

AppCard Injection and Path Optimization: AppCards are designed to inject structured functional semantics into the agent in order to mitigate uncertainty in the decision process. For this case, the Pro Expense home screen is represented with explicit entries for record creation and editing, while the Simple Gallery Pro interface is summarized with functional elements such as thumbnail browsing, image selection, and sharing options. This knowledge is consolidated into a compressed frame that provides the agent with direct mappings between observed states and feasible actions. As a result, the decision sequence is no longer driven by exploratory search but evolves into a stable and logically ordered trajectory. The redundant and error-prone behaviors observed in the baseline are thus eliminated, and the agent follows a path that is both shorter and more predictable.

Analysis: The case study demonstrates how AppCards provide structured knowledge that directly addresses the limitations of baseline agents. By encoding the correct application identity and presenting explicit functional entry points, AppCards resolve entity-level ambiguity and stabilize application selection. By abstracting key visual elements, such as thumbnails, share icons, and form fields, into operational anchors, they transform interface features into consistent functional cues that guide execution. In parallel, AppCards constrain the decision space by reducing an open-ended search into a set of contextually appropriate candidates, aligning

visual recognition with functional reasoning at critical decision points. This combined support allows the agent to transition from fragmented trial and error toward coherent plan execution. The outcome is reflected in shorter trajectories, improved stability in planning, and reliable completion of the cross-application transfer. As illustrated in Figure 5, the contrast between the baseline trajectory and the AppCard-enhanced path highlights the role of structured knowledge in enabling robust and efficient task completion.

6 Discussion

The curiosity mechanism provides a principled means to estimate and quantify an agent’s insufficient understanding during task execution. By formalizing epistemic uncertainty as a numerical score, it allows the differentiation between failures caused by missing functional knowledge and those resulting from alternative factors such as interface variability or execution errors. This diagnostic role supports more accurate interpretation of outcomes and prevents the misattribution of errors to model limitations alone.

In addition to diagnosis, curiosity also drives the automated construction of AppCards. When uncertainty exceeds a threshold, it identifies specific knowledge gaps and triggers targeted retrieval from external sources. The retrieved information is consolidated into structured AppCards that directly address the deficiencies observed during execution. In this way, curiosity transforms external knowledge retrieval into a selective and context-sensitive process, simultaneously explaining failure and guiding the generation of knowledge resources that enhance future performance.

7 Conclusion

This paper presents a curiosity driven knowledge retrieval and augmentation framework for mobile agents in complex applications. Curiosity quantifies epistemic uncertainty during execution and triggers targeted retrieval, while AppCards consolidate external information into structured functional knowledge that can be injected into reasoning and planning. Evaluations on AndroidWorld under a fixed interaction budget and observation modality show consistent gains across backbone models, reaching a new state of the art success rate of 88.8% when combined with GPT 5. The improvements are most pronounced on tasks that require multi step reasoning and cross application coordination, where planning becomes more stable and trajectories are shorter. Limitations include model dependence and the need for calibrated triggering and snippet selection. Future work will explore adaptive thresholding and granularity control, alignment strategies for different backbones, automated construction and versioning of AppCards, and cost aware retrieval scheduling. Overall, the framework provides a practical path toward knowledge aware and robust mobile agents.

Acknowledgments

This work is supported in part by Shanghai Municipal Natural Science Foundation (23ZR1425400) and Shanghai Soft Science Project (25692114700).

References

- [1] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. 2025. Agent s2: A compositional generalist-specialist framework for computer use agents. *arXiv preprint arXiv:2504.00906* (2025).
- [2] DM Anisuzzaman, Jeffrey G Malins, Paul A Friedman, and Zachi I Attia. 2025. Fine-tuning large language models for specialized use cases. *Mayo Clinic Proceedings: Digital Health* 3, 1 (2025), 100184.
- [3] Appium Contributors. 2012. Appium: Cross-platform Mobile UI Automation Framework. <https://github.com/appium/appium>. Version: latest, Accessed: 2025-09-18.
- [4] Richard Bellman. 1957. A Markovian decision process. *Journal of mathematics and mechanics* (1957), 679–684.
- [5] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. Seelick: Harnessing gui grounding for advanced visual gui agents. *arXiv preprint arXiv:2401.10935* (2024).
- [6] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).
- [7] Rumpeng Dai, Linfeng Song, Haolin Liu, Zhenwen Liang, Dian Yu, Haitao Mi, Zhaopeng Tu, Rui Liu, Tong Zheng, Hongtu Zhu, et al. 2025. CDE: Curiosity-Driven Exploration for Efficient Reinforcement Learning in Large Language Models. *arXiv preprint arXiv:2509.09675* (2025).
- [8] DroidRun Team. 2025. DroidRun: A framework for controlling mobile devices through LLM agents. <https://github.com/droidrun/droidrun> MIT License, API and CLI documented in README and docs.
- [9] Xiang Fang, Arvind Easwaran, and Blaise Genest. 2024. Uncertainty-guided appearance-motion association network for out-of-distribution action detection. In *2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR)*. IEEE, 176–182.
- [10] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).
- [11] Teqi Hao, Xiaoyu Tan, Shaojie Shi, Yinghui Xu, and Xihe Qiu. 2025. Reflective Personalization Optimization: A Post-hoc Rewriting Framework for Black-Box Large Language Models. *arXiv preprint arXiv:2511.05286* (2025).
- [12] Junda He, Zhou Yang, Jieke Shi, Chengran Yang, Kisub Kim, Bowen Xu, Xin Zhou, and David Lo. 2024. Curiosity-driven testing for sequential decision-making process. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–14.
- [13] Fuxian Huang, Weichao Li, Jiabao Cui, Yongjian Fu, and Xi Li. 2022. Unified curiosity-driven learning with smoothed intrinsic reward estimation. *Pattern Recognition* 123 (2022), 108352.
- [14] Thanapong Intharath, Michael Firman, and Gabriel J Brostow. 2018. RecurBot: Learn to auto-complete GUI tasks from human demonstrations. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*, 1–6.
- [15] JT-GUIAgent Team. 2025. JT-GUIAgent-V1: A Planner-Grounder Agent for Reliable GUI Interaction. <https://github.com/JT-GUIAgent/JT-GUIAgent> Project description; no public paper available as of now.
- [16] Hanyu Lai, Junjie Gao, Xiao Liu, Yifan Xu, Shudan Zhang, Yuxiao Dong, and Jie Tang. 2025. AndroidGen: Building an Android Language Agent under Data Scarcity. *arXiv preprint arXiv:2504.19298* (2025).
- [17] Guillaume Levy, Cedric Colas, Pierre-Yves Oudeyer, Thomas Carta, and Clement Romac. 2025. WorldLLM: Improving LLMs' world modeling using curiosity-driven theory-making. *arXiv preprint arXiv:2506.06725* (2025).
- [18] Ning Li, Xiangmou Qu, Jiamu Zhou, Jun Wang, Muning Wen, Kounianhua Du, Xingyu Lou, Qiuying Peng, Jun Wang, and Weinan Zhang. 2025. MobileUse: A GUI Agent with Hierarchical Reflection for Autonomous Mobile Operation. *arXiv preprint arXiv:2507.16853* (2025). <https://arxiv.org/abs/2507.16853>
- [19] Ben Liu, Jihai Zhang, Fangquan Lin, Cheng Yang, Min Peng, and Wotao Yin. 2025. Symagent: A neural-symbolic self-learning agent framework for complex reasoning over knowledge graphs. In *Proceedings of the ACM on Web Conference* 2025, 98–108.
- [20] Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Long, Jiadai Sun, Jiaqi Wang, et al. 2024. Autoglm: Autonomous foundation agents for GUIs. *arXiv preprint arXiv:2411.00820* (2024). <https://arxiv.org/abs/2411.00820>
- [21] LX-GUIAgent. 2025. LX-GUIAgent: An advanced GUI Agent developed by China Mobile's LingXi. <https://github.com/LX-GUIAgent/LX-GUIAgent>.
- [22] Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. 2024. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584* (2024).
- [23] Pietro Mazzaglia, Ozan Catal, Tim Verbelen, and Bart Dhoedt. 2022. Curiosity-driven exploration via latent bayesian surprise. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36, 7752–7760.
- [24] Micro Focus. 2000. *SilkTest: Functional and Regression Testing Tool*. Accessed: 2025-09-18.
- [25] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. 2017. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*. PMLR, 2778–2787.
- [26] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. 2025. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326* (2025).
- [27] Christopher Rawles, Sarah Clinckemaeille, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2024. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. *arXiv:2405.14573 [cs.AI]*
- [28] Jürgen Schmidhuber. 1991. A possibility for implementing curiosity and boredom in model-building neural controllers. In *Proc. of the international conference on simulation of adaptive behavior: From animals to animats*, 222–227.
- [29] Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, et al. 2025. A survey on (m) llm-based gui agents. *arXiv preprint arXiv:2504.13865* (2025).
- [30] JT-GUIAgent Team. 2025. JT-GUIAgent: GUI Agent for Android Applications. <https://github.com/JT-GUIAgent/JT-GUIAgent>.
- [31] Bin Xie, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Jie Liu, Min Zhang, and Liqiang Nie. 2025. GUI-explorer: Autonomous Exploration and Mining of Transition-aware Knowledge for GUI Agent. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [32] Yifan Xu, Xiao Liu, Xinghan Liu, Jiaqi Fu, Hanchen Zhang, Bohao Jing, Shudan Zhang, Yuting Wang, Wenyi Zhao, and Yuxiao Dong. 2025. MobileRL: On-line Agentic Reinforcement Learning for Mobile GUI Agents. *arXiv preprint arXiv:2509.18119* (2025).
- [33] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. 2024. Aria-ui: Visual grounding for gui instructions. *arXiv preprint arXiv:2412.16256* (2024).
- [34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- [35] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. 2025. Mobile-Agent-v3: Foundational Agents for GUI Automation. *arXiv preprint arXiv:2508.15144* (2025). <https://arxiv.org/abs/2508.15144>
- [36] Simon Zhai, Hao Bai, Zipeng Lin, Jiayi Pan, Peter Tong, Yifei Zhou, Alane Suhr, Saining Xie, Yann LeCun, Yi Ma, et al. 2024. Fine-tuning large vision-language models as decision-making agents via reinforcement learning. *Advances in neural information processing systems* 37 (2024), 110935–110971.
- [37] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. 2024. Mm-llms: Recent advances in multimodal large language models. *arXiv preprint arXiv:2401.13601* (2024).
- [38] Shanhuai Zhao, Hao Wen, Wenjie Du, Cheng Liang, Yunxin Liu, Xiaozhou Ye, Ye Ouyang, and Yuanchun Li. 2025. LLM-Explorer: Towards Efficient and Affordable LLM-based Exploration for Mobile Apps. *arXiv preprint arXiv:2505.10593* (2025).

A Trajectory Example

To provide a concrete illustration of how our framework executes mobile tasks, we present the complete trajectories of two representative examples that also appear in the main paper. Each trajectory records the full sequence of interface states and actions, together with the final verification outcome.

Expense Add Multiple From Gallery Task

Figure 6 presents the trajectory for the task *Expense Add Multiple From Gallery*. The instruction requires adding all expense records from an image file (expenses.jpg) stored in the gallery into the Pro Expense application. The trajectory demonstrates how the agent switches between applications, selects the correct image, and systematically inputs the extracted expenses. The final verification shows that all three expenses are correctly recorded in Pro Expense, confirming successful task completion.



Figure 6: Trajectory example for the *Expense Add Multiple From Gallery* task. The agent completes the task by importing all expenses from the gallery image into the Pro Expense application.

OsmAnd Marker Task

Figure 7 shows the trajectory for the task *OsmAnd Marker*, where the instruction is to add a location marker for the specified coordinates in the OsmAnd maps application. The trajectory begins with the agent launching the application, navigating through the interface, entering the coordinates, and confirming the marker placement. The final state indicates successful task completion, with the marker correctly added on the map.

B AppCard Specifications

In this appendix we provide representative *AppCards* that were constructed for several applications used in our experiments. Each AppCard encodes the key functionalities of an application into structured modules, serving as reusable building blocks for mobile

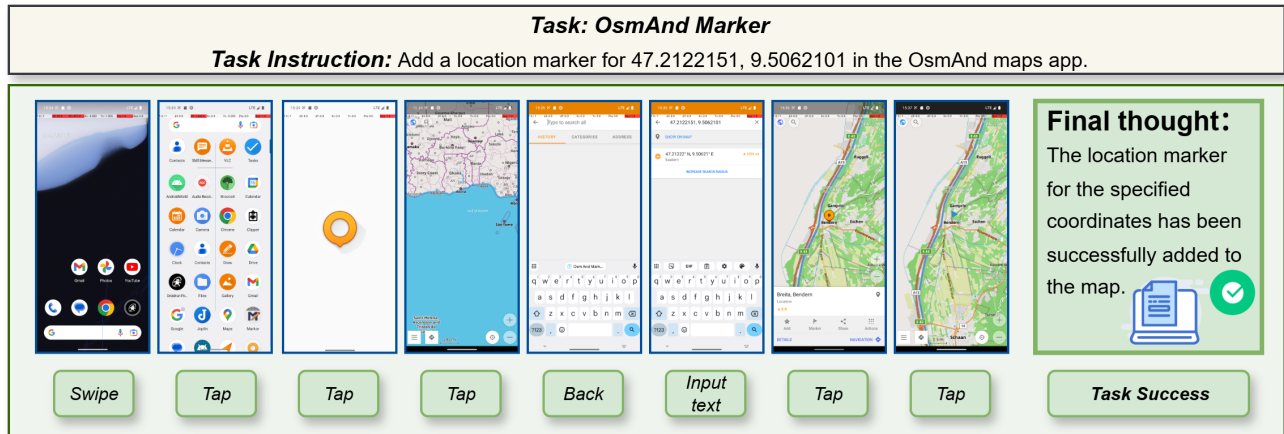


Figure 7: Trajectory example for the *OsmAnd Marker* task. The agent successfully adds the location marker for the specified coordinates.

agent reasoning. The following cases illustrate how task-relevant knowledge is formalized.

Gallery

The *Gallery* application provides image and video management. Its AppCard focuses on viewing, selecting, sharing, and folder navigation.

Gallery:

1. Home: Main screen shows image/video thumbnails in grid; tap any thumbnail to open full view.
2. ViewImage: In full view, swipe left/right to browse; pinch to zoom; toolbar has Share, Edit, Delete.
3. Select: Long-press a thumbnail to select one or multiple images; then actions (Share, Delete, Info) appear in top bar.
4. Share: After selecting image(s), tap 'Share' icon to send to another app.
5. Folders: Accessible via side menu or top bar; lets you switch between Albums/Folders containing media.

VLC

The *VLC* application is a versatile media player supporting local and network playback, streaming, and extensive customization. Its AppCard highlights playback control, media management, and advanced features.

VLC:

1. Open/Play: Open local files, discs, devices, and network streams; plays almost anything.
2. Playback Controls: Play/Pause, seek/jump, next/previous; on-screen and hotkey controls.
3. Subtitles & Audio Tracks: Load/select external subtitle files; switch audio tracks; adjust sync.
4. Playlist: Queue and manage multiple items; reorder and continuous play.
5. Streaming/Convert: Convert/transcode media and stream output to various formats/protocols.

6. Video/Audio Effects: Apply video filters (deinterlace, rotate, crop, etc.) and audio equalizer.
7. Network & Casting: Open network URLs; Chromecast streaming (version 3.0+).
8. High-Res Playback: Hardware-accelerated decoding; up to 4 K/8K, 10-bit, HDR (platform-dependent).
9. Snapshots & Frame Step: Capture still frames (snapshots) and step through frames for inspection.
10. Preferences: Extensive settings for codecs, subtitles, audio/video output, hotkeys, and interfaces.

OsmAnd

The *OsmAnd* application is a navigation and offline maps tool. Its AppCard encodes map viewing, routing, track recording, and plugin extensions.

OsmAnd:

1. **Map Viewing / Offline Maps:** Download maps for regions; view maps offline without internet; support different map styles (topographic, contour, shading).
2. **Search & POI:** Search places by address/name/coordinates; browse POIs (restaurants, ATMs, etc.) and categories.
3. **Navigation / Routing:** Plan routes for car, bicycle, pedestrian; support turn-by-turn guidance; voice prompts; automatic rerouting if deviated.
4. **Track Recording:** Record your trip/track (GPX format); view recorded paths on the map.
5. **Plugins / Extensions:** Use plugin features (e.g. contour lines, hill shading, Mapillary, public transport stops) to extend map display or routing options.
6. **Offline Functionality:** All key features (map view, routing, search) work offline once maps are downloaded.
7. **Settings / Profiles:** Configure vehicle profile (car/bike/pedestrian), map updates, display settings, map styles, routing preferences.

8. ****Map Interactions:**** Zoom, pan, rotate map; set waypoints or intermediate stops; save favorite places.

Broccoli

The *Broccoli* application is a recipe management tool designed for browsing, adding, importing, and assisting in cooking workflows. Its AppCard emphasizes recipe organization, seasonal ingredient guidance, and offline accessibility.

- ```
Broccoli:
```
1. Recipe List / Library: Browse your recipe collection; view recipes by category/hashtag; search by title or keywords.
  2. Add Recipe: Create a new recipe with fields: title, ingredients, instructions/steps; optional image; set category/tags.
  3. Import Recipe: Import recipes from blogs/websites when recipe metadata (JSON-LD) is available; also import from backups.
  4. Cooking Assistant: Use fullscreen mode to follow steps; adjust ingredient quantities; distraction-free cooking view.
  5. Seasonal Calendar: View seasonal ingredients for your region; highlight seasonal items in recipes; search by season.
  6. Backup & Restore / Offline Access: Save recipe backups; restore; access all saved recipes without internet.
  7. Settings & Preferences: Configure theme (e.g. dark mode), units, recipe sorting, display preferences.
  8. Permissions & Storage: Request storage read/write; manage images/photos; ensure data stored on device.

## Contacts

The *Contacts* application provides management of personal and professional information through a structured contact list, detail views, and customizable settings. Its AppCard highlights the main operations including creation, modification, deletion, and search, together with account preferences and permission management that ensure consistent and reliable usage across devices.

- ```
### Contacts:
```
1. Home / Contact List: Displays all contacts; scroll through alphabetical list; tap a contact for details.
 2. Search: Use search bar or search icon from Home to find contacts by name or phone number.
 3. View Contact Details: In contact detail screen, see name, phone numbers, email addresses, photo, other fields.
 4. Add Contact: Tap "+" or "Add" -> input name, phone number, optionally email/address/photo -> save.
 5. Edit Contact: In detail view, tap edit icon -> change fields (name, phone, email, etc.) -> save changes.
 6. Delete Contact: From the detail screen or list (long-press) choose "Delete" -> confirm.
 7. Permissions: Request required permissions (Contacts, Storage if needed for photos).

8. Settings / Default Account: Choose default account for new contacts (if device supports multiple accounts); set display/order preferences.

Pro Expense

The *Pro Expense* application manages personal expense tracking. The AppCard captures core modules ranging from adding and editing expenses to statistics and backup functionality.

- ```
Pro Expense:
```
1. **\*\*Home/Expenses:\*\*** Main list of expense items; tap to view or edit; long-press for actions (Delete, Duplicate).
  2. **\*\*AddExpense (FAB +):\*\*** Create a new expense with fields such as **\*\*Amount\*\***, **\*\*Category\*\***, **\*\*Date/Time\*\***, and **\*\*Note\*\***; save to return to the list.
  3. **\*\*Edit/Delete:\*\*** Open an expense -> tap **\*\*📄\*\*** to edit; use menu/options to **\*\*Delete\*\***.
  4. **\*\*Categories:\*\*** Choose a category when adding/editing an expense; categories power the stats views.
  5. **\*\*Stats:\*\*** View weekly and category statistics (simple graphs/summary) from the stats section.
  6. **\*\*Search/Filter:\*\*** From the list, search by text or filter by date/category to locate expenses.
  7. **\*\*Share-Into (Import Text):\*\*** Receive plain text via Android **\*\*Share\*\*** from other apps (e.g., a note app) to prefill/parse expense content, then confirm to add items.
  8. **\*\*Backup/Restore (Local):\*\*** Manage on-device backups/restore (cloud sync not supported).
  9. **\*\*Settings:\*\*** Adjust language, theme/appearance, and general app preferences (privacy, performance).

## Markor

The *Markor* application is a Markdown-based note-taking and text-editing app. The AppCard highlights its creation, editing, search, and sharing features.

- ```
### Markor:
```
1. Home: List of notes/documents; tap to open folder or file.
 2. CreateNote: Bottom-right '+' -> new note -> editor -> Save.
 3. EditNote: Tap existing note -> modify in Markdown editor -> Save.
 4. DeleteNote: Long-press note -> Delete -> confirm.
 5. Search: Tap search icon -> enter keyword -> view matches.
 6. Settings: Adjust theme, syntax highlighting, font, default folder.
 7. Share: Open note -> share icon -> choose target app.

Together, these AppCards demonstrate how heterogeneous applications can be abstracted into a unified representation that supports modular reasoning and reliable task execution by mobile agents.